

浙江大学实验报告

姓名: _____ 专业: 信息工程 学号: _____
课程名称: 数字系统设计实验 任课老师: 屈民军、唐奕
实验名称: Lab5-常用组合电路模块的设计与应用 实验日期: _____

1 实验目的和要求

1.1. 实验目的

- (1) 掌握用 Verilog HDL 描述数据选择器、加法器和比较器等电路模块。
- (2) 了解"自顶而下"的数字设计方法, 掌握系统层次结构的设计。
- (3) 掌握模块调用的方法, 掌握参数定义和参数传递的方法。
- (4) 掌握 ModelSim 功能仿真的工作流程, 进一步了解 Vivado 的工作流程。
- (5) 认识到文件管理的重要性。

1.2. 实验要求

本次实验分为两个子实验:

(1) 实验一

设计两数之差的绝对值电路: 电路输入 a_{in} 、 b_{in} 为 4 位无符号二进制数, 电路输出 out 为两数之差的绝对值, 即 $out = |a_{in} - b_{in}|$ 。需要调用数据选择器、加法器和比较器等基本模块来设计电路。

(2) 实验二

设计模式比较器电路: 电路的输入为两个 8 位无符号二进制数 a 、 b 和一个模式控制信号 m ; 电路的输出为 8 位无符号二进制数 y 。当 $m=0$ 时, $y=MAX(a,b)$; 而当 $m=1$ 时, 则 $y=MIN(a,b)$ 。需要调用数据选择器、比较器等基本模块来设计电路。

2 设计原理与设计思路

2.1. 基础模块编写

2.1.1. 全加器

全加器完成三个二进制位 (a 、 b 、低位进位 ci) 的求和, 输出和 s 及向高位的进位 co 。真值表推导出 $s = a \oplus b \oplus ci$, $co = (a \& b) \mid (a \& ci) \mid (b \& ci)$ 。因此用 assign 语句直接描述这两个逻辑表达式。代码如下:

```
module full_adder(a,b,ci,s,co);  
    input a,b,ci;  
    output s,co;  
    assign s = a ^ b ^ ci;  
    assign co = (a & b) | (a & ci) | (b & ci);  
endmodule
```

2.1.2. 数字比较器

比较两个 n 位二进制数 a 和 b , 输出三个标志: agb ($a > b$)、 aeb ($a == b$)、 alb ($a < b$)。Verilog 提供了关系运算符和相等运算符, 可以直接写出 $(a > b)$ 、 $(a == b)$ 、 $(a < b)$, 综合工具会自动生成比较逻辑。

```

module mux_2to1(out,in0,in1,addr);
parameter n=1;
output[n-1:0] out;
input[n-1:0] in0,in1;
input addr;

// 2选1逻辑: addr为1时输出in1, 否则输出in0
assign out = addr ? in1 : in0;

endmodule

```

2.1.3. 数据选择器

根据地址信号 addr 选择两个输入之一输出。当 addr=1 时输出 in1，否则输出 in0。用三目运算符?:实现连续赋值。

```

module mux_2to1(out,in0,in1,addr);
parameter n=1;
output[n-1:0] out;
input[n-1:0] in0,in1;
input addr;

// 2选1逻辑: addr为1时输出in1, 否则输出in0
assign out = addr ? in1 : in0;

endmodule

```

2.2. 实验一：设计两数之差的绝对值电路

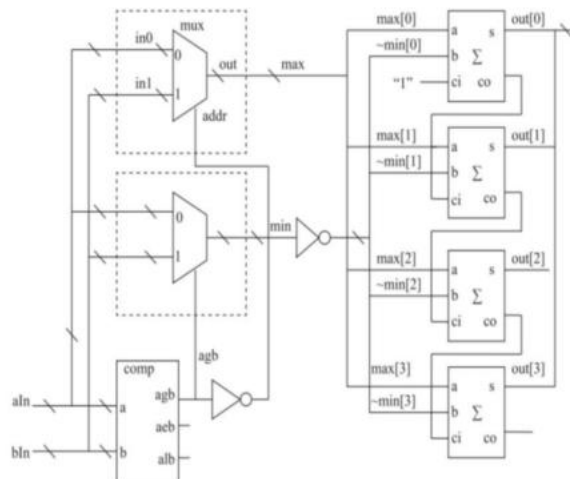
数字电路无法进行减法运算，因此在设计时将减法计算改为补码加一进行计算，但绝对值需要先判断谁大谁小，确保用大数减小数，避免出现负数，即

$$OUT = MAX(aIn, bIn) + (\overline{MIN(aIn, bIn)} + 1)$$

根据本公式，得到设计思路如下：

- (1) 比较大小。通过 comp 模块比较 aIn 和 bIn，输出 agb 信号 (a>b)。如果 agb=1，说明 aIn 大；否则 bIn 大或相等。
- (2) 选出最大值和最小值。利用两个 mux_2to1 选择器。第一个选择器地址取~agb，当 agb=1 时地址为 0，输出 in0 (aIn) 作为最大值；当 agb=0 时地址为 1，输出 in1 (bIn) 作为最大值。第二个选择器地址取 agb，当 agb=1 时输出 in1 (bIn) 作为最小值，否则输出 in0 (aIn) 作为最小值。这样无论输入大小关系如何，max 总保存较大数，min 总保存较小数。
计算差值。由于 $max \geq min$ ，计算 $max - min$ 等价于 $max + (\sim min) + 1$ 。这里使用 4 个全加器构成进位加法器，将 max 的每一位与 ~min 的对应位相加，并且最低位的进位输入 ci 置为 1，实现加 1 操作。最高位的进位输出忽略（因为结果不会超过 4 位）。这样最终输出 out 就是 $max - min$ 的二进制结果，即绝对值。
- (3) ci 置为 1，实现加 1 操作。最高位的进位输出忽略（因为结果不会超过 4 位）。这样最终输出 out 就是 $max - min$ 的二进制结果，即绝对值。

根据电路设计，得到电路原理图及顶层设计 verilog 代码如下：



```

1 // 顶层模块：计算两个4位无符号数的绝对值差 |aIn - bIn|
2 // 实现原理：先比较aIn和bIn，用选择器确定最大值和最小值，再用全加器计算 最大值 + (~最小值) + 1
3 module abs_diff ( aIn, bIn, out );
4     input [3:0] aIn, bIn; // 两个4位输入
5     output [3:0] out; // 4位输出，即|aIn - bIn|
6     // 比较器实例：判断aIn和bIn的大小关系
7     wire agb; // a > b 标志
8     comp #(n(4)) comp_inst (
9         .a(aIn),
10        .b(bIn),
11        .agb(agb), // 输出 a > b
12        .aeb(), // 相等标志未使用
13        .alb() // 小于标志未使用
14    );
15    // 最大值和最小值中间变量
16    wire [3:0] max, min;
17    // 选择器1：根据agb选择最大值
18    // 当agb=1时a>b，max应选aIn；否则max应选bIn
19    // 选择器地址为 ~agb，使得地址=0时选in0(aIn)，地址=1时选in1(bIn)
20    mux_2to1 #(n(4)) mux1 (
21        .out(max),
22        .in0(aIn),
23        .in1(bIn),
24        .addr(~agb)
25    );
26    // 选择器2：根据agb选择最小值
27    // 当agb=1时a>b，min应选bIn；否则min应选aIn
28    // 选择器地址为 agb，使得地址=0时选in0(aIn)，地址=1时选in1(bIn)
29    mux_2to1 #(n(4)) mux2 (
30        .out(min),
31        .in0(aIn),
32        .in1(bIn),
33        .addr(agb)
34    );
35    // 全加器进位链：计算 max - min = max + (~min) + 1
36    // 其中 ~min 是min的按位取反，加法器的最低进位ci设为1
37    wire [2:0] c; // 进位信号
38    // 第0位全加器：a = max[0], b = ~min[0], 进位输入 = 1
39    full_adder adder0 (
40        .a(max[0]),
41        .b(~min[0]),
42        .s(out[0]),
43        .ci(1'b1),
44        .co(c[0])
45    );
46    // 第1位全加器
47    full_adder adder1 (
48        .a(max[1]),
49        .b(~min[1]),
50        .s(out[1]),
51        .ci(c[0]),
52        .co(c[1])
53    );
54    // 第2位全加器
55    full_adder adder2 (
56        .a(max[2]),
57        .b(~min[2]),
58        .s(out[2]),
59        .ci(c[1]),
60        .co(c[2])
61    );
62    // 第3位全加器，进位输出悬空（最高位进位忽略）
63    full_adder adder3 (
64        .a(max[3]),
65        .b(~min[3]),
66        .s(out[3]),
67        .ci(c[2]),
68        .co() // 进位输出不使用
69    );
70 endmodule

```

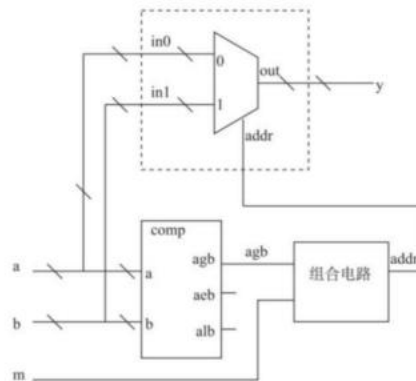
2.3. 实验二：设计模式比较器电路

模式比较器在测试代码中的功能是根据输入 m 的值，输出 a 和 b 的最大值或最小值： $m=0$ 时输出最大值， $m=1$ 时输出最小值。设计思路如下：

- (1) 实例化比较器 `comp` 得到 $a > b$ 的标志 `agb`。
- (2) 利用两个 `mux_2to1` 分别选出最大值和最小值：第一个选择器的地址为 $\sim agb$ ，当 $agb=1$ 时地址为 0，输出 `in0`（即 a ）作为最大值，当 $agb=0$ 时地址为 1，输出 `in1`（即 b ）作为最大值；第二个选择器的地址为 agb ，当 $agb=1$ 时输出 `in1`（即 b ）作为最小值，当 $agb=0$ 时输出 `in0`（即 a ）作为最小值。

(3) 第三个 mux_2to1 根据 m 在最大值和最小值之间选择输出: 地址 m=0 时输出 in0(最大值), 地址 m=1 时输出 in1(最小值)。

根据电路设计, 得到电路原理图及顶层设计 verilog 代码如下:



```

1 // 模式比较器: m=0 输出最大值, m=1 输出最小值
2 module ModeComparator (
3     input  [7:0] a, b, // 两个8位输入
4     input      m,     // 模式选择
5     output [7:0] y     // 输出结果
6 );
7     wire agb; // a > b 标志
8
9     // 比较器: 比较 a 和 b
10    comp #(n(8)) comp_inst (
11        .a(a), .b(b), .agb(agb), .acb(), .alb()
12    );
13
14    wire [7:0] max, min; // 最大值和最小值中间变量
15
16    // 选择器1: 选出最大值, 地址为 ~agb
17    mux_2to1 #(n(8)) mux_max (
18        .out(max), .in0(a), .in1(b), .addr(~agb)
19    );
20
21    // 选择器2: 选出最小值, 地址为 agb
22    mux_2to1 #(n(8)) mux_min (
23        .out(min), .in0(a), .in1(b), .addr(agb)
24    );
25
26    // 选择器3: 根据 m 决定输出最大值还是最小值
27    mux_2to1 #(n(8)) mux_out (
28        .out(y), .in0(max), .in1(min), .addr(m)
29    );
30
31 endmodule

```

3 主要仪器设备

安装了 Modelsim SE 2020.4 和 Vivado 2020.1 的 PC 与 Nexys 实验板一块。

4 实验步骤与操作方法

4.1. 实验一: 设计两数之差的绝对值电路

编写一位全加器、数据选择器、数据比较器以及顶层设计的 Verilog HDL 代码, 并用 ModelSim 软件进行编译, 确保代码功能的正确性, 然后进行波形仿真。

建立 Vivado 工程, 对工程进行综合、引脚约束、实现, 并下载到开发实验板中对设计进行

验证。

4.2. 实验二：设计模式比较器电路

编写顶层设计的 Verilog HDL 代码，调用已有模块，用 ModelSim 软件进行编译，确保代码功能的正确性，然后进行波形仿真。

5 实验数据记录与结果分析

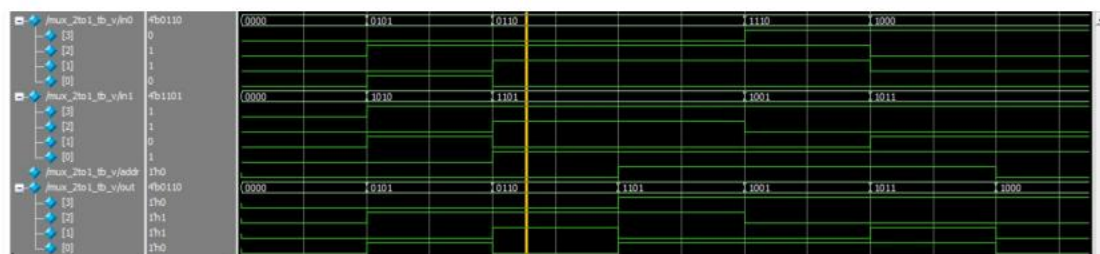
5.1. 基本模块仿真结果

5.1.1. 全加器仿真结果



全加器仿真结果如上图所示，其中五个信号波形从上到下依次对应输入信号 a、输入信号 b、输入进位 ci、输出位 s 和输出进位 co；同一信号仿真波形高位代表 1，低位代表 0。在黄线处，a=1，b=0，ci=0，输出 s=1，co=0，可以验证该全加器的设计是正确的。

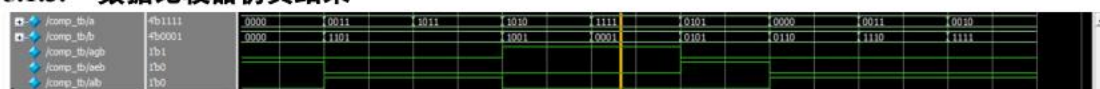
5.1.2. 数据选择器仿真结果



数据选择器的原理是若 addr=1，out=in1；addr=0，out=in0。

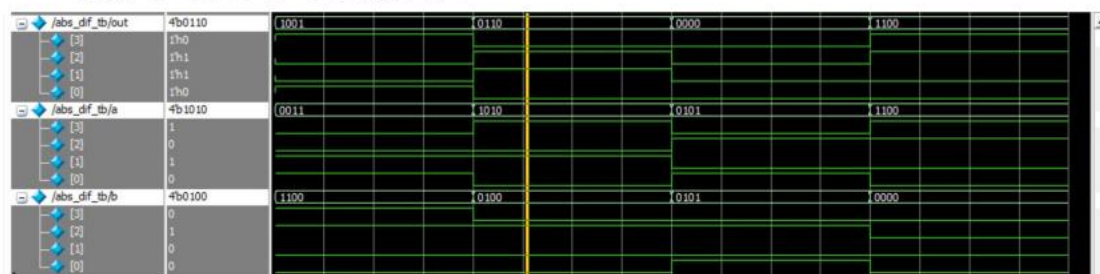
在黄线处，addr=0，in0=0110，in1=1101，发现 out=0110。同理，可验证其它时钟周期仿真结果也符合预期，数据选择器设计正确。

5.1.3. 数据比较器仿真结果



在黄线处，a=1111，b=0001，因此 agb=1，表明 a > b，其他各处结论一致，该仿真结果符合预期，数据比较器设计正确。

5.2. 两数之差的绝对值电路仿真结果

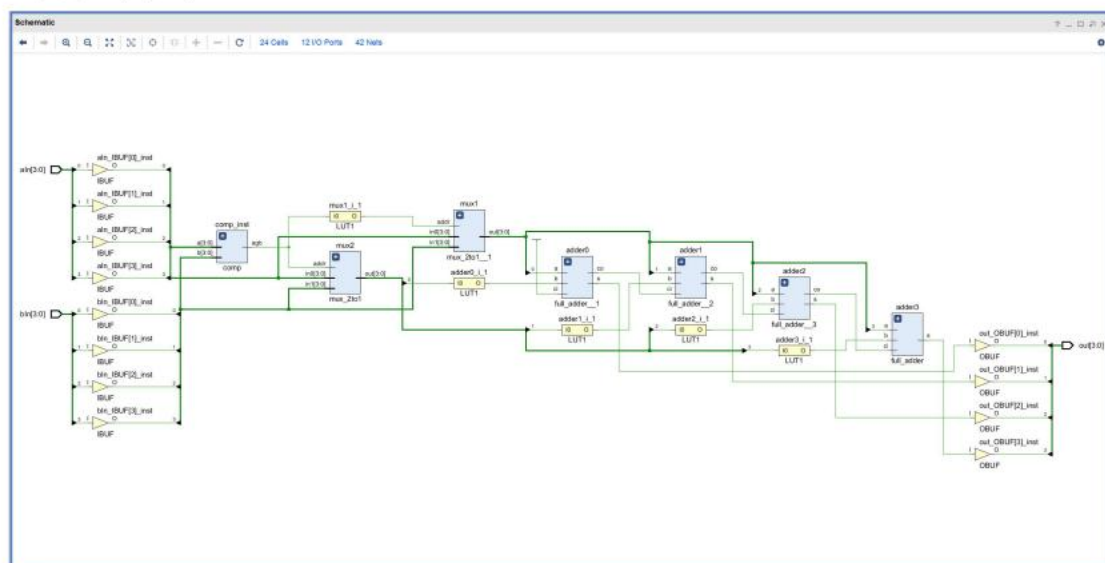


两数之差绝对值电路的顶层模块仿真波形如上图所示。

黄线处设置了 a=1010(10)，b=0100(4)，输出为 out=0110(6)，于是符合求差的绝对值功能，其他各组功能预期一致，顶层设计正确。

Modelsim 的仿真输出结果如下：

工程原理图如下：



本实验没有 CLK 约束。

生成 bitstream 导入开发板，选择对应目标 FPGA 芯片即可实现相应功能。

将体现在演示中。

6 思考题

(1) 求两数之差的绝对值电路，若输入为有符号数（补码），该怎样设计？请画出原理框图并作简要说明。

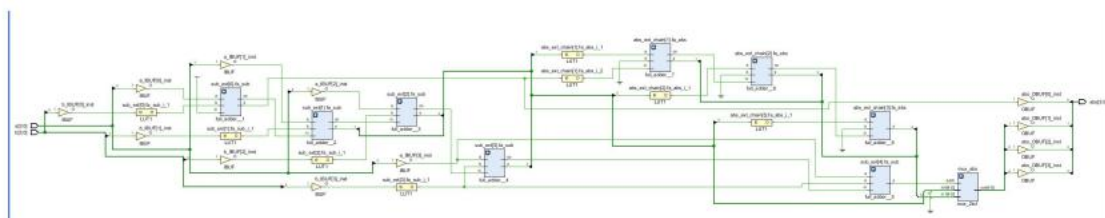
n 位有符号数范围有限（如 4 位为 -8~7）， $A-B$ 可能超出 n 位表示范围（如 $-8-7=-15$ ），直接进行 n 位减法会产生溢出，导致结果错误。因此需要防止溢出。

将 A 和 B 进行符号位扩展至 $n+1$ 位。扩展方法：高位补符号位。这样 $n+1$ 位有符号数范围扩大，能够容纳所有可能的差值（例如 -15 可在 5 位范围内表示）。扩展后计算 $\text{diff} = A_{\text{ext}} - B_{\text{ext}}$ 。

使用全加器链实现减法。减法公式： $A - B = A + (\sim B) + 1$ 。因此对 B_{ext} 取反，然后与 A_{ext} 相加，进位初值设为 1。每个全加器计算一位和及进位，得到差值 diff_ext ($n+1$ 位)。

检查 diff_ext 的符号位。若符号位为 0，绝对值即为 diff_ext ；若符号位为 1，绝对值需要取反加一，即 $\sim \text{diff_ext} + 1$ 。这一步同样用全加器链实现：对 diff_ext 取反，然后加 1，得到 neg_abs_ext 。

使用多路选择器，根据符号位选择 diff_ext 或 neg_abs_ext 作为绝对值结果 abs_ext ($n+1$ 位)。由于绝对值非负，其最高位必然为 0，因此最终输出取低 n 位作为 abs 。



(2) 能否用加法器实现比较器？若能，请画出原理框图。

对于无符号数 A 和 B ，比较大小等价于判断 $A - B$ 是否产生借位。若 $A \geq B$ ，则 $A - B$ 没有借位；若 $A < B$ ，则产生借位。

减法通过加法实现： $A - B = A + (\sim B) + 1$ 。其中 $\sim B$ 是按位取反，+1 通过设置进位初始值为 1 完成。

使用 n 个全加器级联构成减法器：每个全加器计算一位，输入为 $A[i]$ 、 $\sim B[i]$ 和来自低位的进位，输出和与进位。最低位的进位输入设为 1。

最高位全加器产生的进位 c_out 就是无借位标志： $c_out = 1$ 表示 $A \geq B$ （无借位）， $c_out = 0$ 表示 $A < B$ （有借位）。

判断 $A = B$ 需要检查差值是否为 0。差值的每一位由全加器的和输出给出。对 n 位和进行按位或运算，若结果为 0 则说明所有位都是 0，即 $A = B$ 。

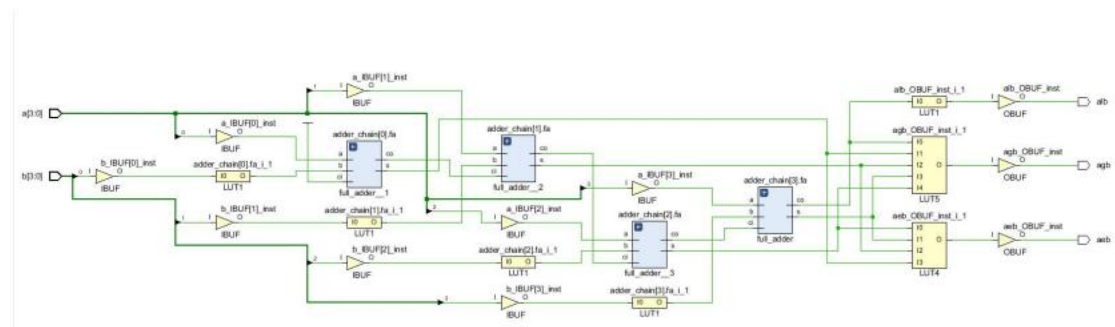
根据以上信息生成三个输出：

$acb = 1$ 当且仅当所有差值的位都是 0。

$agb = 1$ 当 $A \geq B$ 且不相等，即 $c_out = 1$ 且 $acb = 0$ 。

$alb = 1$ 当 $A < B$ ，即 $c_out = 0$ 。

框图如下。



(3) 调用模块时怎样进行参数传递？若模块有多个参数，模块实例的格式如何？

在 Verilog 中，参数传递用于在实例化模块时覆盖模块内部定义的 `parameter` 值。可以在模块名后紧跟 `#( )`，括号内按顺序或按名称列出新的参数值。

对于多个参数，实例化格式如下：

a. 顺序传递（按参数在模块中声明的顺序）

模块名 `#(参数 1 值, 参数 2 值, ...)` 实例名 (端口连接);

b. 名称传递（明确指定参数名）

模块名 `#(参数名 1(值 1), 参数名 2(值 2), ...)` 实例名 (端口连接);

7 总结与心得

本次实验完成了两个组合电路模块的设计与验证，分别是两数之差的绝对值电路和模式比较器电路。通过编写全加器、比较器和数据选择器三个基础模块，并采用自顶向下的层次化设计方法，成功实现了顶层电路。在仿真过程中，使用 ModelSim 对各个模块及整体电路进行了功能验证，波形和打印输出均符合预期。在 Vivado 中对绝对值电路进行了综合、引脚约束、实现并下载到开发板，实际功能正确。通过思考题的讨论，进一步掌握了有符号数绝对值的扩展位宽设计方法以及如何用加法器实现比较器。实验中加深了对 Verilog 参数传递、模块实例化、组合逻辑设计的理解，认识到良好的文件管理和层次化设计对数字系统开发的重要性。

必须承认的是，对于 Verilog HDL 语言的掌握的有限，给实验造成了较多的困难。即使已经设计出原理框图，一开始在编写代码的过程中也常常不能正确的把它转化为对应的 Verilog 代码。其次，在建立工程、添加文件、执行综合等过程中对于 ModelSim 和 Vivado 这两个软件的不熟悉导致要时不时翻阅之前的教材，导致消耗了很多时间。

更为重要的一点是，应用 AI 在解决问题这一方面尤为高效。当遇到一些奇怪的报错而不能及时定位问题原因时，通过 AI 分析错误报告可以帮助理解问题，并成功修改代码错误而不必麻烦老师的帮助。